

Assignment 6: Transformer Fine-tuning

Title

Fine-tuning BERT and Transformer Models for Sentiment Classification

Problem Statement

Master the state-of-the-art in NLP by fine-tuning pre-trained transformer models (BERT, RoBERTa, DistilBERT) for sentiment classification. You will explore transfer learning, compare transformer architectures, visualize attention mechanisms, and achieve superior performance compared to previous RNN-based approaches. This assignment represents cutting-edge NLP techniques used in industry and research.

Dataset

Stanford Sentiment Treebank (SST-2)

- **Source:** [SST-2 on HuggingFace](#) or [Original Stanford Dataset](#)
- **Description:** Movie review sentences with binary sentiment labels
- **Size:** 67k training, 872 validations, 1.8k test samples
- **Alternative:** IMDB dataset for direct comparison with Assignment 5

Detailed Instructions

Part 1: Environment Setup & Data Preparation (10%)

Task 1.1: Install Dependencies

“Bash script code”

```
pip install transformers torch datasets accelerate wandb
```

```
pip install scikit-learn matplotlib seaborn
```

Task 1.2: Load and Explore Dataset

Python template example(for reference) :

```
from datasets import load_dataset
```

```
dataset = load_dataset("sst2")
```

```
train_data = dataset['train']
```

```
val_data = dataset['validation']
```

```
test_data = dataset['test']
```

Analyze dataset:

- Class distribution
- Sentence length statistics
- Sample visualization (10 from each class)

2. Identify challenges (negations, sarcasm, neutral statements)

Task 1.3: Preprocessing for Transformers

- No stemming/lemmatization needed (BERT handles subwords)
- Keep special characters (transformers use them)
- Tokenize using model-specific tokenizer
- Handle maximum sequence length (128, 256, 512)

Part 2: BERT Fine-tuning (30%)

Task 2.1: Load Pre-trained BERT

“Python template example(for reference) “

```
from transformers import BertTokenizer, BertForSequenceClassification

model_name = 'bert-base-uncased'

tokenizer = BertTokenizer.from_pretrained(model_name)

model = BertForSequenceClassification.from_pretrained(
    model_name,
    num_labels=2
)
```

Task 2.2: Tokenization

Python template example(for reference) :

```
def tokenize_function(examples):

    return tokenizer(
        examples['sentence'],
        padding='max_length',
        truncation=True,
        max_length=128,
        return_tensors='pt' )

tokenized_train = train_data.map(tokenize_function, batched=True)
```

Task 2.3: Training Configuration

“ Python template code (for reference) “

```
from transformers import TrainingArguments, Trainer

training_args = TrainingArguments(
    output_dir='./results',
    num_train_epochs=3,
    per_device_train_batch_size=16,
    per_device_eval_batch_size=64,
    warmup_steps=500,
    weight_decay=0.01,
    learning_rate=2e-5,
    logging_dir='./logs',
    logging_steps=100,
    evaluation_strategy="epoch",
    save_strategy="epoch",
    load_best_model_at_end=True,
    metric_for_best_model="accuracy"
)
```

Task 2.4: Define Metrics

“ Python template code (for reference) “

```
from sklearn.metrics import accuracy_score, precision_recall_fscore_support

def compute_metrics(pred):
    labels = pred.label_ids
    preds = pred.predictions.argmax(-1)
    precision, recall, f1, _ = precision_recall_fscore_support(
        labels, preds, average='binary'
    )
    acc = accuracy_score(labels, preds)
    return {
        'accuracy': acc,
        'f1': f1,
        'precision': precision,
```

```
'recall': recall
}
```

Task 2.5: Train and Evaluate

“ Python template code (for reference) “

```
trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=tokenized_train,
    eval_dataset=tokenized_val,
    compute_metrics=compute_metrics
)
trainer.train()
results = trainer.evaluate(tokenized_test)
```

Task 2.6: Hyperparameter Experiments Test different configurations:

1. **Learning rates:** 1e-5, 2e-5, 3e-5, 5e-5
2. **Batch sizes:** 8, 16, 32
3. **Epochs:** 2, 3, 4, 5
4. **Max sequence length:** 128, 256, 512
5. **Warmup steps:** 0, 500, 1000
6. Document impact on performance and training time

Part 3: Compare Multiple Transformer Architectures (25%)

Task 3.1: Fine-tune DistilBERT (Smaller, faster)

“ Python template code (for reference) “

```
from transformers import DistilBertTokenizer, DistilBertForSequenceClassification

model = DistilBertForSequenceClassification.from_pretrained(
    'distilbert-base-uncased',
    num_labels=2
)
```

- 40% fewer parameters than BERT
- 60% faster inference
- Compare accuracy vs speed trade-off

Task 3.2: Fine-tune RoBERTa

“ Python template code (for reference) “

```
from transformers import RobertaTokenizer, RobertaForSequenceClassification

model = RobertaForSequenceClassification.from_pretrained(
    'roberta-base',
    num_labels=2
)
```

- Trained on 10x more data than BERT
- Dynamic masking
- Typically achieves higher accuracy

Task 3.3: Optional: Try ALBERT or ELECTRA

- ALBERT: Parameter-efficient version
- ELECTRA: Discriminative pre-training

Task 3.4: Comprehensive Comparison Create comparison table:

Model	Parameters	Accuracy	F1-Score	Training Time	Inference Time (1k samples)
BERT-base	110M	X%	X	Xm	Xs
DistilBERT	66M	X%	X	Xm	Xs
RoBERTa-base	125M	X%	X	Xm	Xs
LSTM (Assignment 5)	<5M	X%	X	Xm	Xs

Part 4: Attention Visualization (20%)

Task 4.1: Extract Attention Weights

Python code implementation(reference)

```
# Modify model to output attentions

model = BertForSequenceClassification.from_pretrained(
    'bert-base-uncased',
    num_labels=2,
    output_attentions=True
)

# Get attention weights

outputs = model(**inputs)

attentions = outputs.attentions # Tuple of 12 layers
```

Task 4.2: Visualize Self-Attention

1. Visualize attention for sample sentences
2. Create heatmaps showing token-to-token attention
3. Focus on:
 - Last layer attention
 - Attention head analysis (12 heads in each layer)
 - Multi-head attention patterns
4. Use libraries like bertviz

Task 4.3: Interpret Attention Patterns

1. Identify which tokens attend to which
2. Analyze attention for:
 - Negation words ("not", "never")
 - Sentiment words ("excellent", "terrible")
 - Long-range dependencies
3. Compare attention patterns across different models
4. Document interesting findings

Task 4.4: Attention Rollout Aggregate attention across all layers to see information flow from input to output.

Part 5: Advanced Techniques (15%)

Task 5.1: Layer-wise Learning Rate Decay

Python code for reference

Different learning rates for different layers

```
optimizer_grouped_parameters = [  
    {'params': model.bert.embeddings.parameters(), 'lr': 1e-5},  
    {'params': model.bert.encoder.layer[:6].parameters(), 'lr': 2e-5},  
    {'params': model.bert.encoder.layer[6:].parameters(), 'lr': 3e-5},  
    {'params': model.classifier.parameters(), 'lr': 5e-5}  
]
```

Task 5.2: Gradual Unfreezing

Python code for reference

Freeze lower layers initially

```
for param in model.bert.embeddings.parameters():  
    param.requires_grad = False
```

Unfreeze gradually during training

Task 5.3: Data Augmentation

1. Back-translation for data augmentation
2. Synonym replacement using WordNet
3. Random insertion/deletion
4. Evaluate impact on performance

Task 5.4: Ensemble Methods

1. Train 3 different models (BERT, RoBERTa, DistilBERT)
2. Combine predictions:
 - Majority voting
 - Weighted averaging
 - Stacking with meta-learner
3. Measure ensemble performance improvement

Task 5.5: Knowledge Distillation

- Train a smaller student model (e.g., DistilBERT) using logits from a large teacher model (e.g., RoBERTa).
- Compare trade-off: accuracy vs model size vs inference speed.

Task 5.6: Mixed Precision Training (FP16)

- Use Hugging Face accelerate or PyTorch AMP to train with FP16.
- Measure training speed and GPU memory reduction.

Part 6: Deployment (10%)

- Export fine-tuned BERT to **ONNX/TorchScript**.
- Deploy inference with **Streamlit app**:
 - User inputs text → real-time sentiment output with confidence.
 - Visualize attention weights inside app (optional).
- Benchmark inference latency on CPU vs GPU.

Expected Outputs

- Trained models: BERT, DistilBERT, RoBERTa (checkpoints saved).
- Comparison tables (accuracy, training time, inference time).
- Attention heatmaps for at least 5 example sentences.
- Technical report (6–8 pages) with:
 - Model comparisons (RNN vs Transformer).
 - Attention insights.
 - Advanced training tricks impact.
 - Deployment feasibility discussion.

Evaluation Rubric (100 points)

- Model Fine-tuning & Experiments: 30
- Multi-model Comparison: 20
- Attention Visualization & Interpretation: 15
- Advanced Techniques (distillation, FP16, ensemble): 15
- Deployment: 10
- Report Quality: 10

Target Metrics

- BERT baseline: ~90–92% accuracy on SST-2.
- RoBERTa: 92–94% (state-of-the-art).
- DistilBERT: ~88–90% with faster inference.
